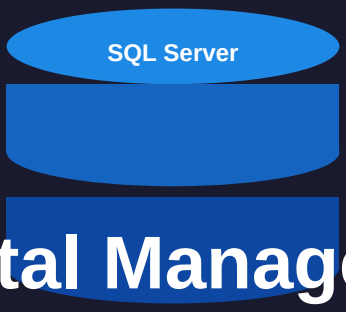


Hospital Management System (HMS)

Database Course Project Report

Submitted By:	**Zain Aftab | FA24-BSE-155**
Group Member:	**Hunain Riasat | FA24-BSE-083**
Course:	**Database Systems**
Instructor:	**Sir Abdul Qayyum**
Project:	**Full-Stack HMS — SQL Server**

NestJS · Prisma ORM · Microsoft SQL Server Express · Next.js 15

Table of Contents

1. Project Overview
2. Technology Stack
3. Database Design
    - 3.1 DBMS — Microsoft SQL Server Express
    - 3.2 Schema Overview — 9 Tables
    - 3.3 Full Table Definitions (DDL)
    - 3.4 Entity Relationships
    - 3.5 Referential Integrity
    - 3.6 CHECK Constraints
    - 3.7 Normalization Analysis
    - 3.8 Migration Strategy
4. Two-Layer Data Validation
5. ORM Layer — Prisma
6. API Layer — NestJS
7. Key Database Operations
8. Authentication & Authorization
9. Data Flow — End-to-End Example
10. Schema Evolution — Migrations
11. Key Design Decisions
12. Summary & Conclusion

1. Project Overview

The Hospital Management System (HMS) is a fully functional clinic management application built as a real-world database course project. It demonstrates end-to-end database integration — from schema design and normalization, through ORM-based query execution, to a live web interface used by multiple user roles.

The system manages the following core hospital operations:

- Patient registration and medical records
- Doctor profiles and scheduling
- Appointment booking and management
- Pharmacy inventory and medicine dispensation
- Billing and invoicing with payment tracking
- Multi-role user access control (RBAC)

2. Technology Stack

Layer	Technology	Purpose
Database	Microsoft SQL Server Express	Relational DBMS — all persistent data
ORM	Prisma 6	Schema definition, migrations, type-safe queries
Backend API	NestJS 10 (Node.js)	RESTful API, business logic, input validation
Frontend	Next.js 15.5 (React 19)	Web UI for all HMS modules
Auth	JWT (JSON Web Tokens)	Stateless authentication, 8-hour expiry
Security	bcryptjs	Secure password hashing — never stored plaintext
Language	TypeScript	End-to-end type safety

3. Database Design

3.1 DBMS — Microsoft SQL Server Express

The project uses SQL Server Express as its RDBMS. The following SQL Server-specific features are employed throughout the schema:

- **NVARCHAR** — Unicode string type supporting multilingual names and addresses
- **DATETIME2** — High-precision datetime for timestamps and appointment dates
- **NVARCHAR(MAX)** — Unbounded text for Base64-encoded patient profile photos
- **INT (cents)** — All monetary values as integers to eliminate floating-point errors
- **Clustered Primary Keys** — PRIMARY KEY CLUSTERED on all tables for efficient lookup
- **Named DEFAULT Constraints** — On status columns, timestamps, and reorder levels
- **FOREIGN KEY with Cascade Rules** — ON DELETE CASCADE and ON DELETE NO ACTION as appropriate
- **CHECK Constraints** — 25 domain integrity rules enforced at the DB engine level

Connection string format:

```
sqlserver://localhost\SQLEXPRESS;database=HospitalDB;
  integratedSecurity=true;trustServerCertificate=true
```

3.2 Schema Overview — 9 Tables

Table Name	Primary Role
User	System users — staff, doctors, admins
Patient	Patient records and medical info
Doctor	Doctor profiles and availability
Appointment	Links patients to doctors on a date/time
Medicine	Pharmacy inventory with stock levels
Invoice	Billing header per patient
InvoiceItem	Individual line items on an invoice
Dispensation	Pharmacy dispense event record
DispensationItem	Individual medicines within a dispense event

3.3 Full Table Definitions (SQL Server DDL)

Each table includes PRIMARY KEY, FOREIGN KEY, DEFAULT, UNIQUE, and CHECK constraints where applicable.

Table: User

```
CREATE TABLE [dbo].[User] (
    [id]           NVARCHAR(191)  NOT NULL,
    [name]         NVARCHAR(191)  NOT NULL,
    [email]        NVARCHAR(191)  NOT NULL,
    [passwordHash] NVARCHAR(191)  NOT NULL,
    [role]         NVARCHAR(50)   NOT NULL,
    [phone]        NVARCHAR(100),
    [department]   NVARCHAR(191),
    [status]       NVARCHAR(50)   NOT NULL DEFAULT 'Active',
    [createdAt]    DATETIME2      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    [updatedAt]    DATETIME2      NOT NULL,
    CONSTRAINT [User_pkey]       PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [User_email_key]  UNIQUE NONCLUSTERED ([email]),
    CONSTRAINT [chk_user_role]   CHECK ([role]   IN ('ADMIN','DOCTOR','RECEPTIONIST','PHARMACIST','STAFF')),
    CONSTRAINT [chk_user_status] CHECK ([status] IN ('Active','Inactive','Suspended'))
);
```

Note: Password hashed with bcryptjs — never stored in plaintext.

Table: Patient

```
CREATE TABLE [dbo].[Patient] (
    [id]               NVARCHAR(191)  NOT NULL,
    [name]             NVARCHAR(191)  NOT NULL,
    [age]              INT            NOT NULL,
    [gender]           NVARCHAR(50)   NOT NULL,
    [contact]          NVARCHAR(100)  NOT NULL,
    [email]            NVARCHAR(191),
    [address]          NVARCHAR(500),
    [disease]          NVARCHAR(191),
    [bloodGroup]       NVARCHAR(20),
    [allergies]        NVARCHAR(500),
    [emergencyContact] NVARCHAR(191),
    [emergencyPhone]   NVARCHAR(100),
    [photoUrl]         NVARCHAR(MAX),      -- Base64-encoded profile photo
    [createdAt]        DATETIME2      NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    [updatedAt]        DATETIME2      NOT NULL,
    CONSTRAINT [Patient_pkey]       PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [chk_patient_age]    CHECK ([age]    >= 0 AND [age] <= 150),
    CONSTRAINT [chk_patient_gender] CHECK ([gender] IN ('Male','Female','Other'))
);
```

Table: Doctor

```
CREATE TABLE [dbo].[Doctor] (
    [id]              NVARCHAR(191)  NOT NULL,
    [name]            NVARCHAR(191)  NOT NULL,
    [age]             INT            NOT NULL,
    [specialization]  NVARCHAR(191)  NOT NULL,
    [qualification]   NVARCHAR(191)  NOT NULL,
    [availability]    NVARCHAR(20)   NOT NULL DEFAULT 'yes',
    [consultationFee] NVARCHAR(50)   NOT NULL,
    [licenseNumber]   NVARCHAR(100),
    [createdAt]       DATETIME2      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    [updatedAt]       DATETIME2      NOT NULL,
    CONSTRAINT [Doctor_pkey]              PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [chk_doctor_age]           CHECK ([age]          >= 18 AND [age] <= 100),
    CONSTRAINT [chk_doctor_gender]        CHECK ([gender]       IN ('Male','Female','Other')),
    CONSTRAINT [chk_doctor_availability]  CHECK ([availability] IN ('yes','no'))
);
```

Table: Appointment

```
CREATE TABLE [dbo].[Appointment] (
    [id]        NVARCHAR(191)  NOT NULL,
    [patientId] NVARCHAR(191)  NOT NULL,
    [doctorId]  NVARCHAR(191)  NOT NULL,
    [date]      DATETIME2      NOT NULL,
    [time]      NVARCHAR(20)   NOT NULL,
    [status]    NVARCHAR(50)   NOT NULL DEFAULT 'Pending',
    [type]      NVARCHAR(50)   NOT NULL DEFAULT 'In-Person',
    [notes]     NVARCHAR(500),
    CONSTRAINT [Appointment_pkey] PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [Appointment_patientId_fkey]
        FOREIGN KEY ([patientId]) REFERENCES [dbo].[Patient]([id]) ON DELETE NO ACTION,
    CONSTRAINT [Appointment_doctorId_fkey]
        FOREIGN KEY ([doctorId])  REFERENCES [dbo].[Doctor]([id])  ON DELETE NO ACTION,
    CONSTRAINT [chk_appt_status]
        CHECK ([status] IN ('Pending','Confirmed','Completed','Cancelled')),
    CONSTRAINT [chk_appt_type]
        CHECK ([type]   IN ('In-Person','Follow-up','Emergency'))
);
```

Table: Medicine

```
CREATE TABLE [dbo].[Medicine] (
    [id]           NVARCHAR(191)  NOT NULL,
    [name]         NVARCHAR(191)  NOT NULL,
    [manufacturer] NVARCHAR(191)  NOT NULL,
    [category]     NVARCHAR(191)  NOT NULL,
    [priceCents]   INT            NOT NULL,  -- Price in cents; no decimals
    [quantity]     INT            NOT NULL,
    [reorderLevel] INT            NOT NULL DEFAULT 20,
    [expiryDate]   DATETIME2,
    [batchNumber]  NVARCHAR(100),
    [unit]         NVARCHAR(50),            -- e.g., Tablet, Syrup, Vial
    CONSTRAINT [Medicine_pkey]        PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [chk_medicine_price]   CHECK ([priceCents]   >= 0),
    CONSTRAINT [chk_medicine_qty]     CHECK ([quantity]     >= 0),
    CONSTRAINT [chk_medicine_reorder] CHECK ([reorderLevel] >= 0)
);
```

Note: Price stored as integer cents (e.g. Rs. 150.00 -> 15000) to eliminate floating-point rounding errors.

Table: Invoice

```
CREATE TABLE [dbo].[Invoice] (
    [id]            NVARCHAR(191)  NOT NULL,
    [invoiceNo]     NVARCHAR(191)  NOT NULL,
    [patientId]     NVARCHAR(191)  NOT NULL,
    [status]        NVARCHAR(50)   NOT NULL DEFAULT 'UNPAID',
    [subtotalCents] INT            NOT NULL DEFAULT 0,
    [discountCents] INT            NOT NULL DEFAULT 0,
    [taxCents]      INT            NOT NULL DEFAULT 0,
    [totalCents]    INT            NOT NULL DEFAULT 0,
    [paymentMode]   NVARCHAR(50),
    CONSTRAINT [Invoice_pkey]          PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [Invoice_invoiceNo_key] UNIQUE NONCLUSTERED ([invoiceNo]),
    CONSTRAINT [Invoice_patientId_fkey]
        FOREIGN KEY ([patientId]) REFERENCES [dbo].[Patient]([id]) ON DELETE NO ACTION,
    CONSTRAINT [chk_invoice_status]
        CHECK ([status]      IN ('DRAFT','UNPAID','PAID')),
    CONSTRAINT [chk_invoice_pay_mode]
        CHECK ([paymentMode] IS NULL OR [paymentMode] IN ('Cash','Card','Online')),
    CONSTRAINT [chk_invoice_total]     CHECK ([totalCents]    >= 0)
);
```

Note: Total formula: totalCents = subtotalCents - discountCents + taxCents

Table: InvoiceItem

```
CREATE TABLE [dbo].[InvoiceItem] (
    [id]             NVARCHAR(191)  NOT NULL,
    [invoiceId]      NVARCHAR(191)  NOT NULL,
    [description]    NVARCHAR(500)  NOT NULL,
    [quantity]       INT            NOT NULL,
    [unitPriceCents] INT            NOT NULL,
    [amountCents]    INT            NOT NULL,  -- quantity x unitPriceCents
    [sourceType]     NVARCHAR(100)  NOT NULL,  -- 'PHARMACY' or 'MANUAL'
    CONSTRAINT [InvoiceItem_pkey] PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [InvoiceItem_invoiceId_fkey]
        FOREIGN KEY ([invoiceId]) REFERENCES [dbo].[Invoice]([id]) ON DELETE CASCADE,
    CONSTRAINT [chk_invitem_qty]       CHECK ([quantity]       >  0),
    CONSTRAINT [chk_invitem_unitprice] CHECK ([unitPriceCents] >= 0),
    CONSTRAINT [chk_invitem_amount]    CHECK ([amountCents]    >= 0)
);
```

Note: ON DELETE CASCADE: deleting an invoice automatically removes all its line items.

Table: Dispensation

```
CREATE TABLE [dbo].[Dispensation] (
    [id]           NVARCHAR(191)  NOT NULL,
    [patientId]    NVARCHAR(191)  NOT NULL,
    [invoiceId]    NVARCHAR(191),           -- Nullable: links to billing
    [pharmacistId] NVARCHAR(191)  NOT NULL, -- User.id of pharmacist
    [totalCents]   INT            NOT NULL,
    [createdAt]    DATETIME2      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT [Dispensation_pkey] PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [Dispensation_patientId_fkey]
        FOREIGN KEY ([patientId]) REFERENCES [dbo].[Patient]([id]) ON DELETE NO ACTION,
    CONSTRAINT [chk_disp_total] CHECK ([totalCents] >= 0)
);
```

Table: DispensationItem

```
CREATE TABLE [dbo].[DispensationItem] (
    [id]             NVARCHAR(191)  NOT NULL,
    [dispensationId] NVARCHAR(191)  NOT NULL,
```

```
    [medicineId]     NVARCHAR(191)  NOT NULL,
    [quantity]       INT            NOT NULL,
    [dosage]         NVARCHAR(100)  NOT NULL,  -- e.g., "1 tablet"
    [frequency]      NVARCHAR(100)  NOT NULL,  -- e.g., "Twice daily"
    [duration]       NVARCHAR(100)  NOT NULL,  -- e.g., "7 days"
    [unitPriceCents] INT            NOT NULL,
    [lineTotalCents] INT            NOT NULL,  -- quantity x unitPriceCents
    CONSTRAINT [DispensationItem_pkey] PRIMARY KEY CLUSTERED ([id]),
    CONSTRAINT [DispensationItem_dispensationId_fkey]
        FOREIGN KEY ([dispensationId]) REFERENCES [dbo].[Dispensation]([id]) ON DELETE CASCADE,
    CONSTRAINT [DispensationItem_medicineId_fkey]
        FOREIGN KEY ([medicineId]) REFERENCES [dbo].[Medicine]([id]) ON DELETE NO ACTION,
    CONSTRAINT [chk_dispitem_qty]       CHECK ([quantity]       >  0),
    CONSTRAINT [chk_dispitem_unitprice] CHECK ([unitPriceCents] >= 0),
    CONSTRAINT [chk_dispitem_linetotal] CHECK ([lineTotalCents] >= 0)
);
```

3.4 Entity Relationships Summary

Relationship	Type	Description
Patient -> Appointment	One-to-Many	A patient can have many appointments
Doctor -> Appointment	One-to-Many	A doctor can have many appointments
Patient -> Invoice	One-to-Many	A patient can have many invoices
Invoice -> InvoiceItem	One-to-Many	An invoice has multiple line items
Patient -> Dispensation	One-to-Many	A patient can have many dispense events
Invoice -> Dispensation	One-to-Many (opt.)	A dispense may link to a billing invoice
Dispensation -> DispItem	One-to-Many	Each dispense has multiple medicine entries
Medicine -> DispItem	One-to-Many	A medicine appears in many dispense items

3.5 Referential Integrity — Cascade Rules

Foreign Key	ON DELETE	ON UPDATE	Reason
InvoiceItem -> Invoice	CASCADE	CASCADE	Items owned by invoice; auto-cleanup on delete
DispItem -> Dispensation	CASCADE	CASCADE	Items owned by dispensation
Appointment -> Patient	NO ACTION	NO ACTION	Prevent accidental patient deletion
Appointment -> Doctor	NO ACTION	NO ACTION	Prevent accidental doctor deletion
Dispensation -> Patient	NO ACTION	NO ACTION	Preserve dispense history
Invoice -> Patient	NO ACTION	CASCADE	Prevent orphaned invoices
DispItem -> Medicine	NO ACTION	CASCADE	Preserve history if medicine ID changes

3.6 CHECK Constraints — Domain Integrity (25 Total)

These constraints fire on every INSERT and UPDATE — even if the application layer is bypassed.

Table	Constraint Name	Rule Enforced
User	chk_user_role	role IN ('ADMIN','DOCTOR','RECEPTIONIST','PHARMACIST','STAFF')

Table	Constraint Name	Rule Enforced
User	chk_user_status	status IN ('Active','Inactive','Suspended')
Patient	chk_patient_age	age >= 0 AND age <= 150
Patient	chk_patient_gender	gender IN ('Male','Female','Other')
Doctor	chk_doctor_age	age >= 18 AND age <= 100
Doctor	chk_doctor_gender	gender IN ('Male','Female','Other')
Doctor	chk_doctor_availability	availability IN ('yes','no')
Appointment	chk_appt_status	status IN ('Pending','Confirmed','Completed','Cancelled')
Appointment	chk_appt_type	type IN ('In-Person','Follow-up','Emergency')
Medicine	chk_medicine_price	priceCents >= 0
Medicine	chk_medicine_qty	quantity >= 0
Medicine	chk_medicine_reorder	reorderLevel >= 0
Invoice	chk_invoice_status	status IN ('DRAFT','UNPAID','PAID')
Invoice	chk_invoice_subtotal	subtotalCents >= 0
Invoice	chk_invoice_discount	discountCents >= 0
Invoice	chk_invoice_tax	taxCents >= 0
Invoice	chk_invoice_total	totalCents >= 0
Invoice	chk_invoice_pay_mode	paymentMode IS NULL OR IN ('Cash','Card','Online')
InvoiceItem	chk_invitem_qty	quantity > 0
InvoiceItem	chk_invitem_unitprice	unitPriceCents >= 0
InvoiceItem	chk_invitem_amount	amountCents >= 0
Dispensation	chk_disp_total	totalCents >= 0
DispensationItem	chk_dispitem_qty	quantity > 0
DispensationItem	chk_dispitem_unitprice	unitPriceCents >= 0
DispensationItem	chk_dispitem_linetotal	lineTotalCents >= 0

3.7 Normalization Analysis — Third Normal Form (3NF)

- **1NF:** All columns hold atomic values. No repeating groups — invoice items and dispensation items are in separate child tables with their own primary keys.
- **2NF:** All non-key attributes fully depend on the primary key. Child tables (InvoiceItem, DispensationItem) have surrogate PKs and depend entirely on them.
- **3NF:** No transitive dependencies. Patient name is stored only in Patient, never repeated in Invoice or Appointment — those reference patientId and JOIN when needed.

Note: Deliberate denormalization: consultationFee stored as a string in Doctor rather than a separate Fee table — acceptable for a single-clinic system.

3.8 Database Migration Strategy

Migration	Change Applied
init_sqlserver	Base schema — all 8 core tables with PKs, FKs, defaults
add_doctors	Doctor table + FK on Appointment
add_appointments	Appointment table
add_user_profile_fields	phone, department, status columns to User
add_clinic_fields	emergencyContact, batchNumber, unit, expiryDate, paymentMode
add_patient_photo	photoUrl NVARCHAR(MAX) to Patient
add-constraints.js	25 CHECK constraints across all tables via idempotent script

Note: Problem: prisma migrate deploy failed on machines with different migration history. Solution: switched to prisma db push which adds only missing columns without checking history.

4. Two-Layer Data Validation

The system enforces data integrity at two independent layers. An invalid value must be rejected by both before reaching the database.

```
User Input (browser)
      |
      v
+----------------------------------+
|  Layer 1 - API Validation        |  class-validator on NestJS DTOs
|  (NestJS + ValidationPipe)       |  Rejects bad input before any DB call
+----------------+-----------------+
                 | valid only
                 v
+----------------------------------+
|  Layer 2 - SQL Server CHECK      |  25 CHECK constraints on all tables
|  (Database Engine)               |  Enforced even if API layer bypassed
+----------------------------------+
```

4.1 API-Level Validation (DTOs)

```
// Patient DTO — validation rules
export class UpsertPatientDto {
  @IsString() @Matches(/^[a-zA-Z][a-zA-Z\s.'-]*$/)
  name!: string;                          // "123" INVALID   "Muhammad Ali" VALID

  @IsInt() @Min(0) @Max(150)
  age!: number;                           // -1 INVALID  200 INVALID  35 VALID

  @IsIn(['Male','Female','Other'])
  gender!: string;

  @Matches(/^\+?[\d\s\-()+]{7,20}$/)
  contact!: string;                       // "abc" INVALID  "0321-1234567" VALID

  @IsOptional() @IsEmail()
  email?: string;

  @IsOptional() @IsIn(['A+','A-','B+','B-','AB+','AB-','O+','O-'])
  bloodGroup?: string;
}
```

4.2 Why Two Layers?

Scenario	Layer 1 (API)	Layer 2 (DB CHECK)
Frontend submits invalid form	Catches it	—
Direct API call via Postman/curl	Catches it	—
API layer is bypassed (direct DB access)	Cannot catch	Catches it
ORM bug passes wrong value	Cannot catch	Catches it
Direct insert via SQL Management Studio	Cannot catch	Catches it

The database-level CHECK constraints act as the **last line of defense** — data integrity is guaranteed regardless of how data enters the system.

5. ORM Layer — Prisma

- **Type-safe queries** — every query result is fully typed in TypeScript
- **Schema-as-code** — schema.prisma is the single source of truth for database structure
- **Auto-generated client** — strongly-typed query builder generated from the schema
- **Transaction support** — complex multi-step operations run atomically via $transaction

```
model Patient {
  id            String        @id @db.NVarChar(191)
  name          String        @db.NVarChar(191)
  age           Int
  gender        String        @db.NVarChar(50)
  photoUrl      String?       @db.NVarChar(Max)
  invoices      Invoice[]
  appointments  Appointment[]
  dispensations Dispensation[]
  createdAt     DateTime      @default(now())
  updatedAt     DateTime      @updatedAt
}
```

Prisma's relation fields (Invoice[], Appointment[]) auto-generate JOIN queries when **include** is used.

6. API Layer — NestJS

6.1 Request Processing Architecture

```
HTTP Request
      |
      v
ValidationPipe  -- rejects invalid input (Layer 1 constraints)
      |
      v
JwtAuthGuard    -- verifies JWT token signature and expiry
      |
      v
RolesGuard      -- checks user role against @Roles() decorator
      |
      v
Controller      -- calls PrismaService methods
      |
      v
PrismaService   -- executes SQL via ORM
      |
```

```
      v
SQL Server      -- enforces CHECK constraints (Layer 2)
```

6.2 REST API Endpoints

Patients (/patients)

Method	Endpoint	Roles Required	SQL Operation
GET	/patients	ADMIN, DOCTOR, RECEPTIONIST, PHARMACIST	SELECT with optional WHERE
POST	/patients	ADMIN, RECEPTIONIST	INSERT INTO Patient
PUT	/patients/:id	ADMIN, RECEPTIONIST	UPDATE Patient WHERE id=?
DELETE	/patients/:id	ADMIN	DELETE FROM Patient WHERE id=?

Doctors, Appointments, Pharmacy, Billing

Module	GET	POST	PUT/PATCH	DELETE
Doctors	ADMIN, DOCTOR, RECEPTIONIST	ADMIN	ADMIN	ADMIN
Appointments	ADMIN, DOCTOR, RECEPTIONIST, PHARMACIST	ADMIN, RECEPTIONIST	ADMIN, RECEPTIONIST, DOCTOR	ADMIN
Pharmacy	ADMIN, PHARMACIST, DOCTOR	ADMIN, PHARMACIST	ADMIN, PHARMACIST	ADMIN
Billing	ADMIN, RECEPTIONIST, PHARMACIST	ADMIN, RECEPTIONIST	ADMIN, RECEPTIONIST	ADMIN

7. Key Database Operations

7.1 Prisma JOIN Query (Include)

```
return this.prisma.invoice.findMany({
  orderBy: { createdAt: 'desc' },
  include: {
    patient: true,   // JOIN Patient ON Invoice.patientId = Patient.id
    items:   true    // JOIN InvoiceItem ON InvoiceItem.invoiceId = Invoice.id
  }
});

-- Generated SQL equivalent:
SELECT i.*, p.*, ii.*
FROM Invoice i
  LEFT JOIN Patient     p  ON i.patientId = p.id
  LEFT JOIN InvoiceItem ii ON ii.invoiceId = i.id
ORDER BY i.createdAt DESC;
```

7.2 Filtered Search Query

```
this.prisma.patient.findMany({
  where: {
    OR: [
      { name:    { contains: search } },
      { id:      { contains: search } },
      { contact: { contains: search } },
      { disease: { contains: search } }
    ]
  },
  orderBy: { name: 'asc' }
```

```
});

-- Generated SQL:
SELECT * FROM Patient
WHERE name LIKE '%search%' OR id LIKE '%search%'
   OR contact LIKE '%search%' OR disease LIKE '%search%'
ORDER BY name ASC;
```

7.3 Atomic Transaction — Medicine Dispensation (7 Steps)

If any step fails, the entire transaction rolls back — no partial data is committed.

```
return this.prisma.$transaction(async (tx) => {
  // 1. Verify patient exists
  const patient = await tx.patient.findUnique({ where: { id: dto.patientId } });
  if (!patient) throw new NotFoundException('Patient not found');

  // 2. Fetch all medicines in one query
  const medicines = await tx.medicine.findMany({
    where: { id: { in: dto.items.map(i => i.medicineId) } }
  });

  // 3. Validate stock (CHECK also enforces quantity >= 0)
  for (const item of dto.items)
    if (item.quantity > medicine.quantity)
      throw new BadRequestException('Insufficient stock');

  // 4. Create Invoice (UNPAID) -> chk_invoice_status enforced
  const invoice = await tx.invoice.create({
    data: { invoiceNo: `INV-${Date.now()}`, patientId: dto.patientId, status: 'UNPAID' }
  });

  // 5. Create Dispensation + DispensationItems
  const dispensation = await tx.dispensation.create({ data: { ... } });

  // 6. Decrement stock -> chk_medicine_qty won't allow negative
  for (const item of dto.items)
    await tx.medicine.update({
      where: { id: item.medicineId },
      data:  { quantity: { decrement: item.quantity } }
    });

  // 7. Insert InvoiceItems + recalculate invoice total
  await tx.invoiceItem.createMany({ data: invoiceLineItems });
  await tx.invoice.update({ where: { id: invoice.id }, data: { totalCents } });

  return { dispensation, invoice };
});
```

8. Authentication & Authorization

8.1 JWT Authentication Flow

```
1. POST /auth/login { email, password }
2. Server: SELECT * FROM [User] WHERE email = ?
3. Server: bcrypt.compare(password, passwordHash)
4. Server: jwt.sign({ sub: id, role, email, name }, secret, { expiresIn: '8h' })
5. Client: stores token in localStorage
6. Every request: Authorization: Bearer <token>
7. JwtAuthGuard validates token signature and expiry
```

```
8. RolesGuard: checks user.role against @Roles(...) on the route handler
```

8.2 Role-Based Access Control (RBAC)

Role	Patients	Doctors	Appointments	Pharmacy	Billing	Users
ADMIN	Full	Full	Full	Full	Full	Full
DOCTOR	Read	—	Read/Update	Read	—	—
RECEPTIONIST	Create/Update	—	Full	—	Full	—
PHARMACIST	Read	—	—	Full	Read/Status	—
STAFF	—	—	Read	—	—	—

9. Data Flow — End-to-End Example

Scenario: Receptionist registers a patient; pharmacist dispenses medicine.

```
1. RECEPTIONIST logs in
   |__ POST /auth/login -> JWT issued with role=RECEPTIONIST

2. POST /patients { name:"Ali Khan", age:35, gender:"Male", contact:"0321-1234567" }
   |__ ValidationPipe: name regex OK  age 0-150 OK  gender IN list OK
   |__ RolesGuard: RECEPTIONIST allowed OK
   |__ INSERT INTO Patient VALUES (...)
   |__ DB CHECK: chk_patient_age OK  chk_patient_gender OK -> committed

3. PHARMACIST logs in, selects patient

4. POST /pharmacy/dispense { patientId, items:[{medicineId, qty:5}] }
   |__ ValidationPipe: qty >= 1 OK
   |__ RolesGuard: PHARMACIST allowed OK
   |__ BEGIN TRANSACTION
      |__ SELECT patient          -> found OK
      |__ SELECT medicines        -> stock validated OK
      |__ INSERT Invoice (UNPAID) -> chk_invoice_status OK
      |__ INSERT Dispensation + DispItems -> chk_dispitem_qty OK
      |__ UPDATE Medicine qty -= 5 -> chk_medicine_qty OK (stays >= 0)
      |__ INSERT InvoiceItems
      |__ UPDATE Invoice totalCents -> chk_invoice_total OK
   |__ COMMIT

5. RECEPTIONIST marks invoice PAID
   |__ PATCH /billing/{id}/status { status:'PAID', paymentMode:'Cash' }
   |__ UPDATE Invoice SET status='PAID' -> chk_invoice_status OK
```

10. Schema Evolution — Migrations

Migration File	Change	SQL Added
init_sqlserver	Base schema — 8 core tables	Full CREATE TABLE statements
add_doctors	Doctor table + FK on Appointment	CREATE TABLE Doctor, ALTER TABLE Appointment ADD CONSTRAIN
add_appointments	Appointment table	CREATE TABLE Appointment
add_user_profile_fields	phone, department, status to User	ALTER TABLE [User] ADD [phone] NVARCHAR(100) etc.

Migration File	Change	SQL Added
add_clinic_fields	emergencyContact, batchNumber, unit	Multiple ALTER TABLE ... ADD statements
add_patient_photo	Profile photo upload support	ALTER TABLE Patient ADD [photoUrl] NVARCHAR(MAX)
add-constraints.js	25 CHECK constraints on all tables	ALTER TABLE ... ADD CONSTRAINT ... CHECK (...)

11. Key Database Design Decisions

Decision	Alternative	Chosen	Reason
Monetary values	DECIMAL(10,2)	INT cents	Eliminates floating-point errors
Photo storage	File system / Blob	Base64 NVARCHAR(MAX)	No external storage for single-server clinic
InvoiceItem table	JSON column in Invoice	Normalized child table	3NF; enables per-item queries
Disp. vs Invoice	Single combined table	Linked separate tables	Different lifecycles
Validation layers	DB constraints only	API DTOs + SQL CHECK	API gives better errors; DB is final safety net
Schema sync	prisma migrate deploy	prisma db push	Handles drift across team machines
String IDs	Auto-increment INT	Manual string IDs	Human-readable IDs (P001, D002)
Named CHECKs	Unnamed constraints	Named (chk_*)	Identifiable in error messages

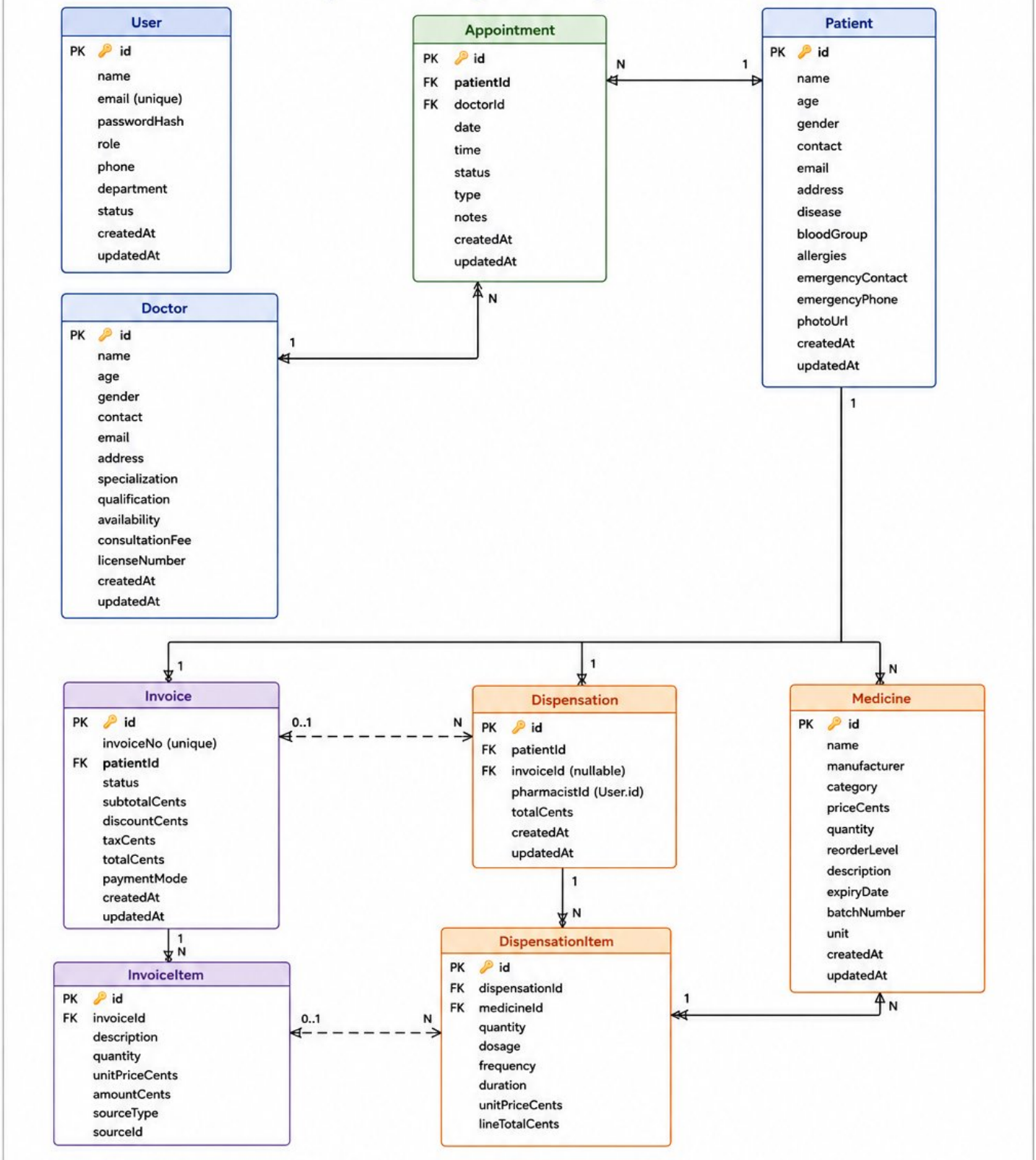
12. Summary & Conclusion

This project demonstrates comprehensive application of database concepts in a production-quality system, going well beyond basic CRUD to implement enterprise-grade features:

Topic	Implementation
Schema Design	9 normalized tables (3NF) with PKs, FKs, defaults, and unique constraints
SQL Server Features	NVARCHAR, DATETIME2, INT cents, clustered PKs, named constraints, cascades
CHECK Constraints	25 domain-integrity rules across all tables enforced at the DB engine level
Two-Layer Validation	class-validator DTOs at API layer + CHECK constraints at DB layer
ORM	Prisma — type-safe abstraction with schema-as-code and auto-generated client
Transactions	7-step atomic dispensation operation with automatic rollback on failure
Migrations	Version-controlled schema evolution across 7 SQL migration files
RBAC	5-role access control enforced at both API (JWT + RolesGuard) and frontend
Referential Integrity	Cascade deletes, no-action constraints, unique constraints across all relations
Query Patterns	JOIN (include), filtered LIKE/OR search, bulk update with decrement, aggregation

The system is fully operational: it connects to a live SQL Server Express instance, enforces data integrity through both API validation and database-level constraints, executes complex atomic transactions, and serves a multi-role React frontend — all launched via a single START_HMS.bat command.

HMS — Hospital Management System | Database ERD



LEGEND (CROW'S FOOT)

- Mandatory One
- Optional One (0..1)
- Many (0..*)
- Optional Many (0..*)

CARDINALITY

- 1 = One
- N = Many
- 0..1 = Zero or One
- 0..* = Zero or Many

RELATIONSHIP SUMMARY

1. Patient —< Appointment
   One Patient has Many Appointments
2. Doctor —< Appointment
   One Doctor has Many Appointments
3. Patient —< Invoice
   One Patient has Many Invoices
4. Invoice —< InvoiceItem (ON DELETE CASCADE)
   One Invoice has Many Line Items
5. Patient —< Dispensation
   One Patient has Many Dispensations
6. Invoice —o< Dispensation (NULLABLE)
   One Invoice optionally linked to Many Dispensations
7. Dispensation —< DispensationItem
   One Dispensation has Many Items (ON DELETE CASCADE)
8. Medicine —< DispensationItem
   One Medicine appears in Many Dispensation Items

NOTES

- User table is standalone; pharmacistId in Dispensation is a soft reference to User.id.
- InvoiceItem.sourceId optionally references Dispensation.id when sourceType = PHARMACY.
- ON DELETE CASCADE applied on:
  - Invoice → InvoiceItem
  - Dispensation → DispensationItem
- All prices are stored in cents (paisa) to avoid rounding issues.

COLOR CODE (ENTITY GROUPS)

- Core People (User, Patient, Doctor)
- Scheduling (Appointment)
- Billing (Invoice, InvoiceItem)
- Pharmacy (Medicine, Dispensation, DispensationItem)